

DevOps Case Study – Report

Task 0 – Kubernetes Deployment (bắt buộc)

Thư mục k8s/ gồm: configmap.yaml (namespace + config), deployment.yaml, service.yaml, ingress.yaml (optional), hpa.yaml.

kubectl vs Helm — vì sao chọn kubectl?

Tiêu chí	kubectl apply (đã chọn)	Helm
Độ phức tạp	Thấp, minh bạch, dễ review	Cao hơn (chart, template, values)
Phù hợp	1 app, ít môi trường — đúng scope case study	Nhiều app/nhiều env, cần packaging & versioning release
Templating	Không (raw manifest)	Mạnh (values theo env)
Rollback	<code>kubectl rollout undo</code>	<code>helm rollback</code> (theo release revision)

Với phạm vi một service, kubectl đủ và rõ ràng. Khi hệ thống mở rộng thành nhiều microservice với nhiều môi trường (như bối cảnh đề bài), Helm (hoặc Kustomize) là bước tiến hợp lý để quản lý biến theo env và versioning release.

Rolling update & rollback khi deploy lỗi

Deployment dùng strategy: RollingUpdate với maxUnavailable: 0, maxSurge: 1: tạo pod mới, chờ pod mới Ready rồi mới hạ pod cũ → không mất service. Nếu pod mới không đạt readiness, rollout status timeout và pipeline gọi:

```
kubectl -n demo rollout undo deployment/demo-app
kubectl -n demo rollout status deployment/demo-app
```

Scale application

- Thủ công:** `kubectl scale deployment/demo-app --replicas=4`
- Tự động:** hpa.yaml scale theo CPU (target 70%), min 2 – max 6 replicas (cần metrics-server).

Các vấn đề thường gặp khi deploy K8s

Triệu chứng	Nguyên nhân thường gặp	Cách xử lý
ImagePullBackOff	Sai tag/registry, thiếu imagePullSecret, image chưa load vào cluster local	Kiểm tra tag; kind load / minikube image load; thêm secret
CrashLoopBackOff	App crash lúc start, sai env/config	kubectl logs, kiểm tra ConfigMap/Secret, liveness quá gắt
Pod restart liên tục	Liveness probe fail vì initialDelaySeconds quá ngắn	Tăng initial delay / dùng readiness cho warm-up
Pod Pending	Không đủ CPU/mem, thiếu node	Xem kubectl describe pod (Events), chỉnh requests/limits
Service không nhận traffic	Sai selector/label, readiness fail	Đối chiếu selector với label pod; check endpoints

TASK 1 - Thiết kế CI/CD Pipeline

Flow: Checkout → Test & Quality (parallel) → Build Docker → Push Image → Deploy K8s → Smoke Test

Nếu bất kỳ stage nào sau Deploy fail (điển hình là Smoke Test), khối post { failure } sẽ chạy kubectl rollout undo để quay về revision trước — rollback tự động.

Điểm thiết kế chính

- **Parallel stages:** Unit Test và Lint/Audit chạy song song → rút ngắn thời gian pipeline.
- **Image tag theo commit SHA:** mỗi build là image bất biến, truy vết được, rollback chính xác thay vì dùng latest.
- **Error handling:** timeout toàn pipeline, disableConcurrentBuilds, rollout status --timeout để deploy không treo vô hạn.
- **Secrets:** registry credential & kubeconfig lấy qua Jenkins Credentials (withCredentials), không hardcode; login bằng --password-stdin.
- **Deploy an toàn:** kubectl apply -f k8s/ rồi set image → Kubernetes tự rolling update theo strategy đã khai báo.

Rollback strategy

Hai lớp: (1) tự động qua post { failure } gọi rollout undo; (2) thủ công — vì mỗi image có tag SHA riêng, có thể pin lại tag cũ bằng kubectl set image. Kubernetes giữ lịch sử revision (rollout history) nên rollback là thao tác một dòng lệnh.

Task 4 – Pipeline Optimization (phân tích)

Scenario: build ~25 phút, Docker build hay fail, dev than deploy chậm.

Phân tích nguyên nhân

- Không cache dependency → mỗi build cài lại toàn bộ package.
- Dockerfile không tối ưu layer → đổi 1 dòng code rebuild cả image.
- Các bước tuần tự (test, build, scan) đáng lẽ chạy song song.
- Docker build fail do thiếu retry / phụ thuộc mạng khi kéo base image.

5 đề xuất tối ưu

#	Giải pháp	Tác động
1	Dependency caching: cache ~/.npm / node_modules theo hash package-lock.json	Bỏ hẳn cài lại khi deps không đổi
2	Multi-stage + tách layer deps (đã áp dụng trong app/Dockerfile)	Chỉ rebuild layer thay đổi
3	Parallel stages (đã áp dụng): test // lint/audit	Giảm tổng thời gian pipeline
4	BuildKit + cache mount (--mount=type=cache), bật DOCKER_BUILDKIT=1	Build nhanh, ổn định hơn
5	Artifact reuse: build image một lần, promote cùng image qua staging → prod	Nhất quán + tiết kiệm thời gian

Troubleshooting & phần được mock

Vấn đề	Xử lý
Jenkins không build được image	Kiểm tra mount /var/run/docker.sock; user jenkins thuộc nhóm docker
Agent không kết nối	Đặt đúng AGENT_SECRET (sinh khi tạo node); hoặc chạy pipeline trên controller
Prometheus không thấy Jenkins	Cài plugin Prometheus metrics để có endpoint /prometheus
Smoke test fail ở local	Query svc.cluster.local chỉ resolve trong cluster; local dùng port-forward + curl